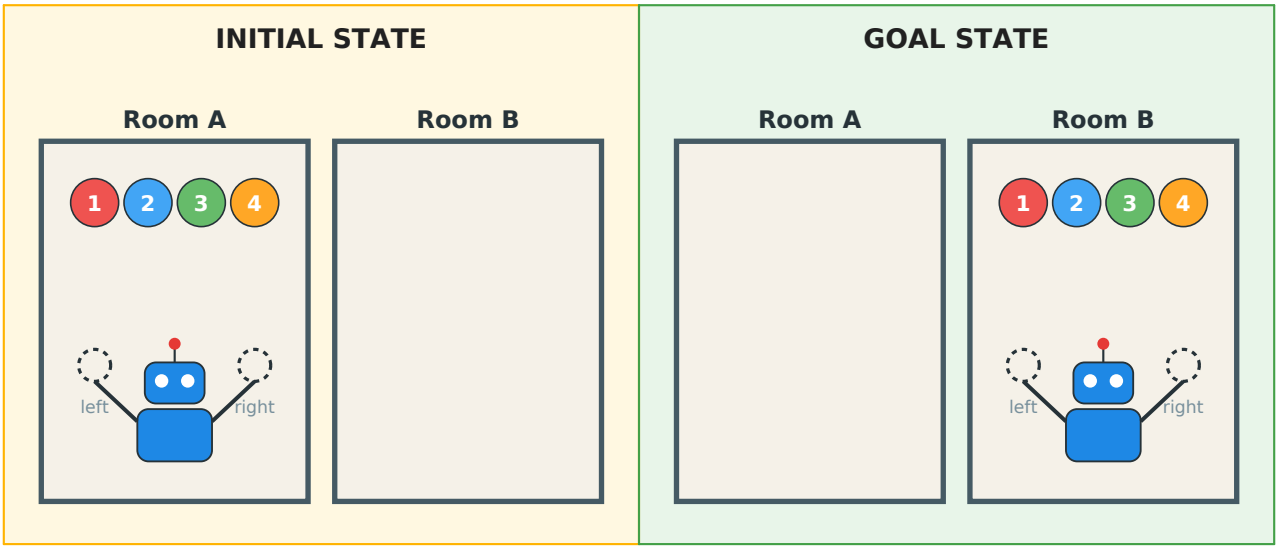
Problem 3: Gripper

Level: **Basic** · Folder: **03_Gripper/** · Files: domain.pddl, problem.pddl, solution_plan.txt

1. The problem

A robot with **two hands** (left and right grippers) is in room A together with four balls. It can pick up a ball with a free gripper, carry it while moving between rooms, and drop it. The goal is to have **all four balls in room B**.



The goal picture shows one state that satisfies the goal (the final state of the plan). Only the facts in the :goal section are required; anything else may be different.

What you will learn

- Several **types** in one domain (room, ball, gripper).
- Actions with **three parameters**, and how the planner *grounds* them.
- A **resource**-like predicate: `(free ?g)` says a hand is available.
- How a planner finds an efficient plan (using both hands to halve the trips).

2. Formulating the problem in PDDL

Step 1: Types and objects. room: roomA, roomB. ball: ball1 to ball4. gripper: left, right.

Step 2: Predicates describe what can be true or false in a state:

Predicate	Meaning
`(robot-at ?r)`	The robot is in room ?r.
`(at ?b ?r)`	Ball ?b lies on the floor of room ?r.
`(free ?g)`	Gripper ?g holds nothing.
`(carry ?b ?g)`	Gripper ?g holds ball ?b.

Step 3: Actions. Each action has *parameters*, a *precondition* (what must be true to apply it) and an *effect* (facts that become **true (+)** or **false (−)**):

Action	Precondition	Effect
move `?from - room, ?to - room` Move the robot between rooms (it carries whatever its grippers hold).	`(robot-at ?from)`	`+ (robot-at ?to)` `− (robot-at ?from)`

pick
`?b - ball, ?r - room, ?g - gripper`
Pick ball ?b from the floor of room ?r with free gripper ?g.
	`(at ?b ?r)`
	`(robot-at ?r)`
	`(free ?g)`
		`+ (carry ?b ?g)`
		`− (at ?b ?r)`
		`− (free ?g)`

drop
`?b - ball, ?r - room, ?g - gripper`
Drop ball ?b held by gripper ?g in the current room ?r.
	`(carry ?b ?g)`
	`(robot-at ?r)`
		`+ (at ?b ?r)`
		`+ (free ?g)`
		`− (carry ?b ?g)`

Step 4: Initial state (`:init`) lists *every* fact that is true at the start. Everything else is false (closed-world assumption). **Step 5: Goal** (`:goal`) lists the facts that must be true at the end.

INITIAL STATE (7 facts)	GOAL

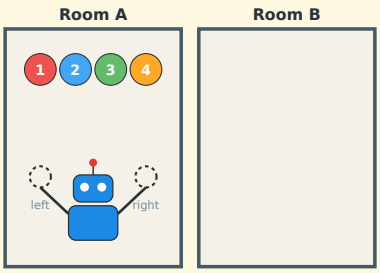
`(robot-at roomA)`
`(free left)`
`(free right)`
`(at ball1 roomA)`
`(at ball2 roomA)`
`(at ball3 roomA)`
`(at ball4 roomA)`
	`(at ball1 roomB)`
	`(at ball2 roomB)`
	`(at ball3 roomB)`
	`(at ball4 roomB)`

Modelling notes

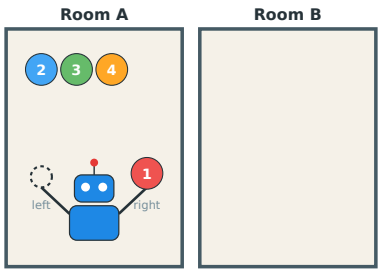
- **Grounding:** the planner creates every combination of objects for each action. For **pick** that is 4 balls x 2 rooms x 2 grippers = 16 ground actions. Most of them are not applicable in a given state.
- A ball that is carried is **not** at any room. pick deletes `(at ?b ?r)`. The ball's location is then implied by the robot's location.
- The optimal plan is 11 steps: two round trips with two balls each, and the final trip has no return.

3. Solution plan (11 actions, optimal)

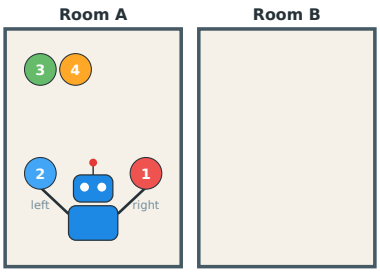
Look at how both grippers are filled before every trip. The pictures below show the state **after** each action. They were produced by executing the plan on the PDDL model.



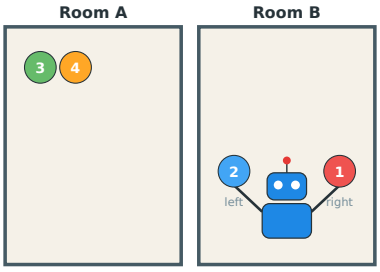
Initial state



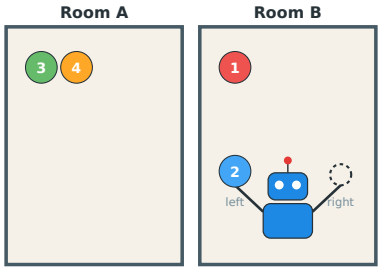
Step 1
`(pick ball1 rooma right)`



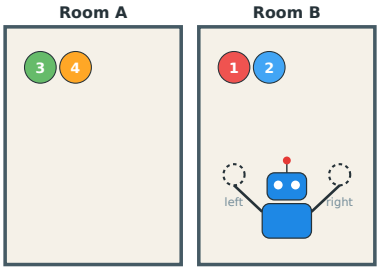
Step 2
`(pick ball2 rooma left)`



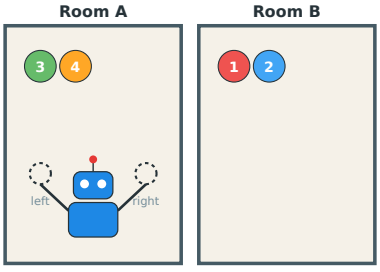
Step 3
`(move rooma roomb)`



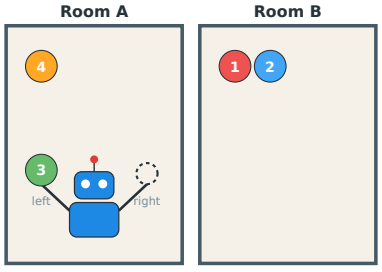
Step 4
`(drop ball1 roomb right)`



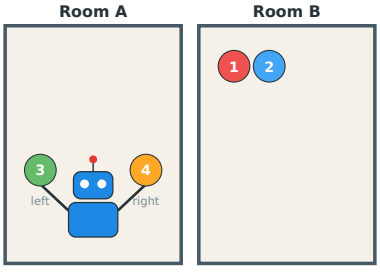
Step 5
`(drop ball2 roomb left)`



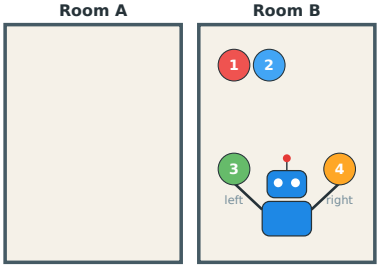
Step 6
`(move roomb rooma)`



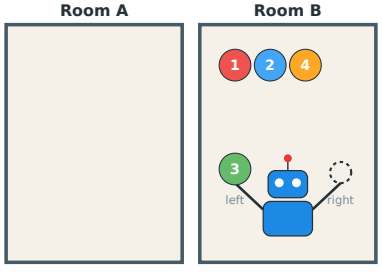
Step 7
`(pick ball3 rooma left)`



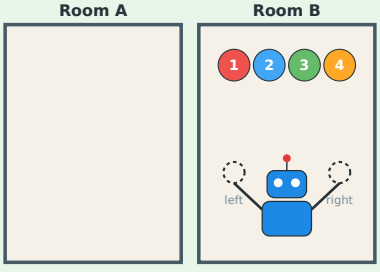
Step 8
`(pick ball4 rooma right)`



Step 9
`(move rooma roomb)`



Step 10
`(drop ball4 roomb right)`



Step 11
`(drop ball3 roomb left)`

4. How to run it

Online (no installation)	Open **editor.planning.domains**, paste *domain.pddl* and *problem.pddl* into two tabs and press **Solve**.
pyperplan (Python)	`pip install pyperplan` `pyperplan domain.pddl problem.pddl` (breadth-first search) `pyperplan -s astar -H lmcut domain.pddl problem.pddl` (optimal A*)
Fast Downward	`fast-downward.py domain.pddl problem.pddl --search "astar(lmcut())"`

5. Lab exercises

1. Use 6 balls. Predict the optimal plan length before running the planner.
2. Add a third gripper *middle*. How does the plan length change?
3. Add room C and make the goal: ball1, ball2 in room B and ball3, ball4 in room C.
4. The move action does not check that ?from and ?to differ. Is that a problem? Why or why not?

6. PDDL source code (appendix)

domain.pddl

```
;; ============================================================
;; Problem 3 : Gripper
;; Level     : Basic
;; A robot with two hands (grippers) carries balls between rooms.
;; ============================================================
(define (domain gripper)
  (:requirements :strips :typing)

  (:types room ball gripper)

  (:predicates
    (robot-at ?r - room)            ; robot is in room ?r
    (at ?b - ball ?r - room)        ; ball ?b is in room ?r
    (free ?g - gripper)             ; gripper ?g is empty
    (carry ?b - ball ?g - gripper)  ; gripper ?g holds ball ?b
  )

  (:action move
    :parameters (?from - room ?to - room)
    :precondition (robot-at ?from)
    :effect (and (robot-at ?to)
                 (not (robot-at ?from)))
  )

  (:action pick
    :parameters (?b - ball ?r - room ?g - gripper)
    :precondition (and (at ?b ?r)
                       (robot-at ?r)
                       (free ?g))
    :effect (and (carry ?b ?g)
                 (not (at ?b ?r))
                 (not (free ?g)))
  )

  (:action drop
    :parameters (?b - ball ?r - room ?g - gripper)
    :precondition (and (carry ?b ?g)
                       (robot-at ?r))
    :effect (and (at ?b ?r)
                 (free ?g)
                 (not (carry ?b ?g)))
  )
)
```

problem.pddl

```
;; Four balls are in room A. Move all of them to room B.
;; The robot has a left and a right gripper (2 balls per trip).
(define (problem gripper-4-balls)
  (:domain gripper)

  (:objects
    roomA roomB         - room
    ball1 ball2 ball3 ball4 - ball
    left right          - gripper
  )

  (:init
    (robot-at roomA)
    (free left) (free right)
    (at ball1 roomA) (at ball2 roomA)
    (at ball3 roomA) (at ball4 roomA)
  )

  (:goal (and (at ball1 roomB) (at ball2 roomB)
              (at ball3 roomB) (at ball4 roomB)))
)
```

solution_plan.txt

```
 1. (pick ball1 rooma right)
 2. (pick ball2 rooma left)
 3. (move rooma roomb)
 4. (drop ball1 roomb right)
 5. (drop ball2 roomb left)
 6. (move roomb rooma)
 7. (pick ball3 rooma left)
 8. (pick ball4 rooma right)
 9. (move rooma roomb)
10. (drop ball4 roomb right)
11. (drop ball3 roomb left)
```